

FASE 0

FUNDACAO TECNICA

Base de projeto, dados e qualidade antes de qualquer feature complexa

Criar uma base previsivel para que o Codex consiga evoluir o jogo sem misturar regra, conteudo e interface, reduzindo retrabalho quando o projeto passar de prototipo local para multiplayer online.

Documento detalhado para planejamento, implementacao e validacao no Codex

1. Objetivo da fase

Estabelecer repositório, stack, convenções, modelo de dados, testes, configuração de RNG e ferramentas básicas de desenvolvimento. A fase não entrega gameplay completo; ela entrega a estrutura que torna as próximas fases seguras e repetíveis.

Resultado esperado: Um comando sobe o projeto, um comando roda os testes e um personagem de teste pode ser carregado de arquivos de dados, validado e exibido em um diagnóstico simples.

2. Contexto e dependências

Como o jogo terá engine de combate, simulador, cliente web e futuramente servidor autoritativo, a lógica central deve nascer desacoplada da interface. O maior risco nesta fase é escolher uma estrutura que obrigue reescrever regras quando o multiplayer chegar.

- Documento de Decisões de Projeto v0.1 como fonte de regras.
- MVP 0.1 definido como validação de combate, sem login, economia ou arte final.
- Ambiente local capaz de executar ferramentas do projeto, testes e build web.

3. Escopo incluído

- Repositório organizado por domínios: engine, conteúdo/dados, aplicação web, testes e ferramentas.
- Stack web leve e adequada a browser/mobile, com TypeScript ou linguagem equivalente fortemente tipada.
- Modelo de dados externo ao código para classes, skills, armas, armaduras, acessórios, consumíveis e inimigos.
- IDs estáveis e versionamento de schema/conteúdo.
- Módulo de RNG injetável, com suporte a seed fixa.
- Testes unitários básicos e validação de schemas.
- Scripts padrão: instalar, desenvolver, testar, lintar, buildar e executar simulações futuras.
- Convenções de commit/branch/issues e um README operacional curto para o Codex.

4. Fora do escopo

- Login e autenticação.
- Banco de dados remoto.
- Servidor multiplayer.
- Arte final, áudio ou animações definitivas.
- Economia, ranking, drops reais ou monetização.
- Otimizar prematuramente para escala de produção.

5. Requisitos funcionais e técnicos

Arquitetura de código

- Separar engine de combate de React/UI ou framework visual.
- Nenhuma fórmula de combate deve depender do DOM, canvas ou rede.
- Criar interfaces claras para Character, Attributes, EquipmentLoadout e GameDataRegistry.
- Evitar singletons globais de estado que dificultem testes e simulação em lote.

Dados de jogo

- Conteúdo deve ser declarativo em JSON/TS data/estrutura equivalente.
- Cada entidade deve ter id imutável, nome de exibição, versão e parâmetros.
- Validar referências: skill inexistente, item com atributo desconhecido e status inválido devem falhar cedo.
- Permitir que números sejam alterados sem editar a engine.

RNG e reprodutibilidade

- Centralizar qualquer aleatoriedade em um serviço de RNG.
- Permitir seed manual em testes.

- Registrar seed usada em cada simulacao.
- Proibir uso disperso de random nativo dentro das regras.

Qualidade e automacao

- Lint e formatacao automatica.
- Testes unitarios executaveis em CI/local.
- Type-check obrigatorio antes de merge.
- Fixtures pequenas para personagens e equipamentos de teste.

6. Fluxos principais

Carregamento de dados

1. Aplicacao inicia.
2. Registry carrega arquivos de conteudo.
3. Schemas e referencias sao validados.
4. Erros impedem inicializacao e apontam arquivo/id problematizado.
5. Personagem fixture e montado usando apenas IDs validos.

Execucao local

1. Desenvolvedor/Codex instala dependencias.
2. Executa comando de desenvolvimento.
3. Abre pagina de diagnostico simples.
4. Roda testes sem passos manuais adicionais.

7. Plano de implementacao

Marco	Objetivo	Saida verificavel
0.1 Estrutura	Criar pastas, tooling, scripts e build minimo.	Projeto sobe localmente e gera build de producao.
0.2 Schemas	Definir dados de personagem, atributos, equipamento e conteudo.	Fixtures validam e erros de schema sao legiveis.
0.3 RNG/Testes	Criar RNG seeded e harness de testes.	A mesma seed gera a mesma sequencia.
0.4 Guardrails	Documentar regras para o Codex e adicionar lint/type-check.	PR local falha quando viola checagens basicas.

8. Testes e validacao

Unitarios

- Validacao de schema de cada tipo de conteudo.
- IDs duplicados e referencias quebradas falham.
- RNG seeded e deterministico.
- Serializacao/deserializacao de fixture preserva dados.

Smoke tests

- Instalacao limpa -> test -> build.
- Aplicacao abre sem erros no browser desktop e viewport mobile basico.

9. Metricas e diagnostico

- Tempo de instalacao/build apenas como diagnostico.

- Cobertura focada nos módulos de schema e RNG, sem perseguir percentual artificial.
- Número de dependências e tamanho do bundle registrados para evitar crescimento acidental desde o início.

Definition of Done

- Repositório limpo e documentado.
- Engine pode ser importada sem importar UI.
- Conteúdo básico e validado por schema.
- RNG seeded disponível.
- Testes, lint, type-check e build passam em um comando previsível.
- Nenhuma feature fora do escopo foi acoplada à fundação.

Riscos e mitigações

Risco	Impacto	Mitigação
Overengineering	Alto	Criar apenas abstrações exigidas pelas fases 1-3; evitar ECS, microserviços ou infra distribuída sem necessidade.
Stack inadequada ao servidor futuro	Alto	Manter engine pura e serializável; decidir framework visual sem prender a regra.
Dados hardcoded	Medio	Code review deve rejeitar números de conteúdo dentro de componentes/UI.
RNG não reproduzível	Alto	Toda fonte aleatória passa por interface única e seed registrada.

Decisões em aberto nesta fase

Tema	Decisão pendente	Como resolver
Stack	Framework web e empacotamento mobile futuro.	Prototipar build mínima e avaliar suporte, bundle e ergonomia.
Formato de dados	JSON validado, TS data ou formato equivalente.	Escolher o que der melhor equilíbrio entre tipagem e edição por conteúdo.

Entregáveis

- Repositório/configuração inicial.
- README de desenvolvimento.
- Schemas e fixtures.
- Módulo RNG seeded.
- Suite básica de testes.
- Página simples de diagnóstico de carregamento de dados.

Gate para avançar

Gate: Um comando sobe o projeto; testes executam; dados de um personagem de teste podem ser carregados e validados sem depender da interface final.

Com a fundação aprovada, a Fase 1 implementa a matemática e as entidades reais do Combat Engine.

Orientação de execução no Codex

- Trabalhar em issues pequenas com critério de aceite objetivo.
- Não adicionar biblioteca sem justificar o problema que resolve.

- Para cada novo tipo de dado, adicionar schema + fixture valida + fixture invalida.
- Manter engine importavel em testes Node/headless.
- Fixar versao do runtime e lockfile para que o ambiente seja reproduzivel entre maquinas.
- Qualquer novo schema deve entrar com fixture valida, fixture invalida e teste de referencia quebrada.
- Manter IDs de conteudo estaveis; renomear texto de exibicao nao deve alterar identidade interna.
- Quando uma regra aparecer hardcoded em UI ou teste, mover o parametro para o registro de dados antes de expandi-la.
- Revisar dependencias novas pelo impacto em bundle, manutencao e compatibilidade com execucao headless.
- Registrar no README o comando exato usado pelo Codex para test, lint, type-check e build.